MACHINE LEARNING- ITA0613

LAB EXPERIMENTS

Experiment 1: FIND-S Algorithm

Aim

To implement the Find-S Algorithm in Python to find the most specific hypothesis from the given training data.

Algorithm

Step 1: Import the pandas library.

Step 2: Read the training dataset (FindS.csv).

Step 3: Display the dataset.

Step 4: Separate the attributes (input) and target (output).

Step 5: Initialize the hypothesis with '0' for all attributes.

Step 6: Read each training example one by one.

Step 7: Check whether the target value is "Yes".

- If Yes, compare each attribute with the current hypothesis.
- If the hypothesis value is '0', replace it with the current attribute value.
- If the attribute values are different, replace that position with '?'.

Step 8: Ignore all "No" examples.

Step 9: Display the final most specific hypothesis.

Coding

 # FIND-S ALGORITHM

```
import pandas as pd

# Read the CSV file

data = pd.read_csv(r"C:\Users\divya\Downloads\FindS.csv")

# Display the dataset

print("Training Data:\n")

print(data)

# Separate attributes and target
```

```
concepts = data.iloc[:, :-1].values
target = data.iloc[:, -1].values
# Initialize the hypothesis
hypothesis = ['0'] * len(concepts[0])
# Apply Find-S Algorithm
for i in range(len(concepts)):
    if target[i] == "Yes":
        for j in range(len(hypothesis)):
            if hypothesis[j] == '0':
                hypothesis[j] = concepts[i][j]
            elif hypothesis[j] != concepts[i][j]:
                hypothesis[j] = '?'
# Display the final hypothesis
print("\nMost Specific Hypothesis:")
print(hypothesis)
```

Output

```
Training Data:

     Sky  Temp Humidity    Wind Water Forecast EnjoySport
0  Sunny  Warm   Normal  Strong  Warm     Same        Yes
1  Sunny  Warm     High  Strong  Warm     Same        Yes
2  Rainy  Cold     High  Strong  Warm   Change         No
3  Sunny  Warm     High  Strong  Cool   Change        Yes

Most Specific Hypothesis:
['Sunny', 'Warm', '?', 'Strong', '?', '?']
```

Result

Thus, the Find-S Algorithm was implemented successfully, and the most specific hypothesis was obtained from the given training data.

Experiment 2: Candidate Elimination Algorithm

Aim

To implement the Candidate Elimination Algorithm in Python to find the set of all hypotheses that are consistent with the given training data.

Algorithm

Step 1: Import the pandas library.

Step 2: Read the training dataset (CandidateElimination.csv).

Step 3: Display the dataset.

Step 4: Separate the attributes (input) and target (output).

Step 5: Initialize the Specific Hypothesis (S) with the first training example.

Step 6: Initialize the General Hypothesis (G) with '?'.

Step 7: Read each training example one by one.

- If the example is positive (Yes):
    - Update the Specific Hypothesis.
    - Remove inconsistent values from the General Hypothesis.
- If the example is negative (No):
    - Update the General Hypothesis by specializing it to exclude the negative example.

Step 8: Remove duplicate or empty hypotheses.

Step 9: Display the final Specific Hypothesis and General Hypothesis.

Coding

```
# CANDIDATE ELIMINATION ALGORITHM

import pandas as p

# Read the CSV file

data = pd.read_csv("CandidateElimination.csv")

# Display the dataset

print("Training Data:\n")

print(data)

# Separate attributes and target

concepts = data.iloc[:, :-1].values
```

```
target = data.iloc[:, -1].values
# Initialize Specific Hypothesis
specific = concepts[0].copy()
# Initialize General Hypothesis
general = [['?' for i in range(len(specific))] for j in range(len(specific))]
# Apply Candidate Elimination Algorithm
for i in range(len(concepts)):
    if target[i] == "Yes":
        for j in range(len(specific)):
            if concepts[i][j] != specific[j]:
                specific[j] = '?'
                general[j][j] = '?'
    elif target[i] == "No":
        for j in range(len(specific)):
            if concepts[i][j] != specific[j]:
                general[j][j] = specific[j]
            else:
                general[j][j] = '?'
# Remove empty hypotheses
general = [g for g in general if g != ['?'] * len(specific)]
# Display the result
print("\nSpecific Hypothesis:")
print(specific)
print("\nGeneral Hypothesis:")
for g in general:
    print(g)
```

Output

```
Training Data:

     Sky  Temp Humidity    Wind Water Forecast EnjoySport
0  Sunny  Warm   Normal  Strong  Warm     Same        Yes
1  Sunny  Warm     High  Strong  Warm     Same        Yes
2  Rainy  Cold     High  Strong  Warm   Change         No
3  Sunny  Warm     High  Strong  Cool   Change        Yes

Specific Hypothesis:
['Sunny' 'Warm' '?' 'Strong' '?' '?']

General Hypothesis:
['Sunny', '?', '?', '?', '?', '?']
['?', 'Warm', '?', '?', '?', '?']
```

Result

Thus, the Candidate Elimination Algorithm was implemented successfully, and the Specific Hypothesis and General Hypothesis consistent with the given training data were obtained.

Experiment 3: ID3 Decision Tree Algorithm

Aim

To implement the ID3 (Iterative Dichotomiser 3) Decision Tree Algorithm in Python and classify a new sample using the given training dataset.

Algorithm

Step 1: Import the required libraries.

Step 2: Read the dataset (ID3.csv).

Step 3: Display the dataset.

Step 4: Convert the categorical values into numerical values using LabelEncoder.

Step 5: Separate the input attributes (X) and target output (y).

Step 6: Create the Decision Tree Classifier using the entropy criterion (ID3).

Step 7: Train the model using the training data.

Step 8: Select a sample for testing.

Step 9: Predict the class label of the sample.

Step 10: Display the predicted result.

Coding

```
# ID3 DECISION TREE ALGORITHM

import pandas as pd

from sklearn.preprocessing import LabelEncoder

from sklearn.tree import DecisionTreeClassifier

# Read the CSV file

data = pd.read_csv("ID3.csv")

# Display the dataset

print("Training Data:\n")

print(data)

# Convert categorical data into numerical data

encoder = LabelEncoder()

encoded_data = data.copy()

for column in encoded_data.columns:
```

```
    encoded_data[column] = encoder.fit_transform(encoded_data[column])
# Separate input and output
X = encoded_data.iloc[:, :-1]
y = encoded_data.iloc[:, -1]
# Create Decision Tree using Entropy (ID3)
model = DecisionTreeClassifier(criterion="entropy")
# Train the model
model.fit(X, y)
# Test with the first sample
sample = X.iloc[[0]]
prediction = model.predict(sample)
print("\nPrediction for First Sample:")
print(prediction)
```

Output

```
Training Data:

    Outlook Temperature Humidity    Wind Play
0     Sunny         Hot     High    Weak   No
1     Sunny         Hot     High  Strong   No
2  Overcast         Hot     High    Weak  Yes
3      Rain        Mild     High    Weak  Yes
4      Rain        Cool   Normal    Weak  Yes
5      Rain        Cool   Normal  Strong   No
6  Overcast        Cool   Normal  Strong  Yes
7     Sunny        Mild     High    Weak   No
8     Sunny        Cool   Normal    Weak  Yes
9      Rain        Mild   Normal    Weak  Yes

Prediction for First Sample:
[0]
```

Result

Thus, the ID3 Decision Tree Algorithm was implemented successfully, and the model correctly classified the given sample using the training dataset.

Experiment 4: Artificial Neural Network using Backpropagation Algorithm

Aim

To build an Artificial Neural Network (ANN) by implementing the Backpropagation Algorithm in Python and test it using the given dataset.

Algorithm

Step 1: Import the required libraries.

Step 2: Read the dataset (Backpropagation.csv).

Step 3: Display the dataset.

Step 4: Convert the categorical values into numerical values using LabelEncoder.

Step 5: Separate the input attributes (X) and target output (y).

Step 6: Split the dataset into training data and testing data.

Step 7: Create an Artificial Neural Network (MLP Classifier).

Step 8: Train the model using the training data.

Step 9: Test the model using the testing data.

Step 10: Display the predicted output and accuracy.

Coding

```
# BACKPROPAGATION ALGORITHM

import pandas as pd

from sklearn.preprocessing import LabelEncoder

from sklearn.model_selection import train_test_split

from sklearn.neural_network import MLPClassifier

from sklearn.metrics import accuracy_score

# Read the CSV file

data = pd.read_csv("Backpropagation.csv")

# Display the dataset

print("Training Data:\n")

print(data)

# Convert categorical data into numerical data

encoder = LabelEncoder()
```

```
encoded_data = data.copy()
for column in encoded_data.columns:
    encoded_data[column] = encoder.fit_transform(encoded_data[column])
# Separate input and output
X = encoded_data.iloc[:, :-1]
y = encoded_data.iloc[:, -1]
# Split the dataset
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=1
)
# Create ANN Model
model = MLPClassifier(hidden_layer_sizes=(5,), max_iter=1000, random_state=1)
# Train the model
model.fit(X_train, y_train)
# Predict the output
prediction = model.predict(X_test)
# Display results
print("\nPredicted Output:")
print(prediction)
print("\nActual Output:")
print(y_test.values)
# Display accuracy
accuracy = accuracy_score(y_test, prediction)
print("\nAccuracy:", accuracy)
```

output

```
Training Data:

    Outlook Temperature Humidity    Wind Play
0     Sunny         Hot     High    Weak   No
1     Sunny         Hot     High  Strong   No
2  Overcast         Hot     High    Weak  Yes
3      Rain        Mild     High    Weak  Yes
4      Rain        Cool   Normal    Weak  Yes
5      Rain        Cool   Normal  Strong   No
6  Overcast        Cool   Normal  Strong  Yes
7     Sunny        Mild     High    Weak   No
8     Sunny        Cool   Normal    Weak  Yes
9      Rain        Mild   Normal    Weak  Yes
```

```
Predicted Output:
[1 1 1]

Actual Output:
[1 1 1]

Accuracy: 1.0
```

Result

Thus, the Artificial Neural Network using the Backpropagation Algorithm was implemented successfully, and the model was trained and tested using the given dataset. The predicted output and accuracy were obtained successfully.

Experiment 7: Logistic Regression (LR)

Aim

To implement the Logistic Regression Algorithm in Python and classify the given dataset.

Algorithm

1. Import the required libraries.
2. Read the dataset (LogisticRegression.csv).
3. Display the dataset.
4. Convert categorical data into numerical values.
5. Separate the input and output.
6. Split the dataset into training and testing data.
7. Create the Logistic Regression model.
8. Train the model.
9. Predict the output.
10. Display the accuracy.

Coding

```
import pandas as pd
from sklearn.preprocessing import LabelEncoder
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score
# Read CSV file
data = pd.read_csv("LogisticRegression.csv")
print("Training Data:\n")
print(data)
encoder = LabelEncoder()
encoded_data = data.copy()
for column in encoded_data.columns:
    encoded_data[column] = encoder.fit_transform(encoded_data[column])
X = encoded_data.iloc[:, :-1]
```

```
y = encoded_data.iloc[:, -1]
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=1
)
model = LogisticRegression()
model.fit(X_train, y_train)
prediction = model.predict(X_test)
print("\nPredicted Output:")
print(prediction)
print("\nActual Output:")
print(y_test.values)
print("\nAccuracy:", accuracy_score(y_test, prediction))
```

output

```
Training Data:

    Outlook Temperature Humidity    Wind Play
0     Sunny         Hot     High    Weak   No
1     Sunny         Hot     High  Strong   No
2  Overcast         Hot     High    Weak  Yes
3      Rain        Mild     High    Weak  Yes
4      Rain        Cool   Normal    Weak  Yes
5      Rain        Cool   Normal  Strong   No
6  Overcast        Cool   Normal  Strong  Yes
7     Sunny        Mild     High    Weak   No
8     Sunny        Cool   Normal    Weak  Yes
9      Rain        Mild   Normal    Weak  Yes

Predicted Output:
[1 1 1]

Actual Output:
[1 1 1]

Accuracy: 1.0
```

Result

Thus, the Logistic Regression algorithm was implemented successfully and the accuracy was obtained.

Experiment 8: Linear Regression

Aim

To implement the Linear Regression Algorithm in Python.

Algorithm

1. Import the required libraries.
2. Read the dataset (LinearRegression.csv).
3. Display the dataset.
4. Separate input and output.
5. Create the Linear Regression model.
6. Train the model.
7. Predict the output.
8. Display the predicted result.

Coding

```
import pandas as pd
from sklearn.linear_model import LinearRegression
# Read CSV file
data = pd.read_csv("LinearRegression.csv")
print("Training Data:\n")
print(data)
X = data[['Hours']]
y = data['Marks']
model = LinearRegression()
model.fit(X, y)
prediction = model.predict([[7]])
print("\nPredicted Marks for 7 Hours:")
print(prediction)
```

Output

```
Training Data:

   Hours  Marks
0      1     15
1      2     25
2      3     35
3      4     45
4      5     55
5      6     65
6      7     75
7      8     85
8      9     95
9     10    100

Predicted Marks for 7 Hours:
[74.09090909]
```

Result

Thus, the Linear Regression algorithm was implemented successfully and the predicted output was obtained.

Experiment 9: Compare Linear and Polynomial Regression

Aim

To compare the performance of Linear Regression and Polynomial Regression using Python.

Algorithm

1. Import the required libraries.
2. Read the dataset (PolynomialRegression.csv).
3. Separate input and output.
4. Train the Linear Regression model.
5. Convert input into polynomial features.
6. Train the Polynomial Regression model.
7. Predict using both models.
8. Compare the outputs.

Python Code

```
import pandas as pd
from sklearn.linear_model import LinearRegression
from sklearn.preprocessing import PolynomialFeatures
# Read CSV file
data = pd.read_csv("PolynomialRegression.csv")
print("Training Data:\n")
print(data)
X = data[['Experience']]
y = data['Salary']

linear = LinearRegression()
linear.fit(X, y)
poly = PolynomialFeatures(degree=2)
X_poly = poly.fit_transform(X)
poly_model = LinearRegression()
poly_model.fit(X_poly, y)
```

```
linear_prediction = linear.predict([[6]])

poly_prediction = poly_model.predict(poly.transform([[6]]))

print("\nLinear Regression Prediction:")

print(linear_prediction)

print("\nPolynomial Regression Prediction:")

print(poly_prediction)
```

Output:

```
Training Data:

   Experience  Salary
0           1   25000
1           2   27000
2           3   32000
3           4   40000
4           5   52000
5           6   68000
6           7   87000
7           8  110000
8           9  138000
9          10  170000


Linear Regression Prediction:
[82866.66666667]

Polynomial Regression Prediction:
[67806.06060606]
```

Result

Thus, the Linear Regression and Polynomial Regression models were implemented successfully and their predictions were compared.

Experiment 10: Expectation Maximization (EM) Algorithm

Aim

To implement the Expectation Maximization Algorithm using the Gaussian Mixture Model in Python.

Algorithm

1. Import the required libraries.
2. Read the dataset (ExpectationMaximization.csv).
3. Display the dataset.
4. Select the input features.
5. Create the Gaussian Mixture Model.
6. Train the model.
7. Predict the cluster labels.
8. Display the cluster labels.

Python Code

```
import pandas as pd
from sklearn.mixture import GaussianMixture
# Read CSV file
data = pd.read_csv("ExpectationMaximization.csv")
print("Training Data:\n")
print(data)
X = data[['Height', 'Weight']]
model = GaussianMixture(n_components=2, random_state=1)
model.fit(X)
prediction = model.predict(X)
print("\nCluster Labels:")
print(prediction)
```

Output:

```
Training Data:

   Height  Weight
0     150      45
1     152      47
2     155      50
3     157      52
4     160      55
5     162      58
6     165      60
7     167      63
8     170      66
9     172      68

Cluster Labels:
[0 0 0 0 0 1 0 1 1 1]
```

Result

Thus, the Expectation Maximization algorithm was implemented successfully, and the data points were grouped into clusters successfully.

11. Credit Score Classification

Aim

To implement a **classification model** to predict whether a person's credit score is good or poor based on financial attributes.

Algorithm / Procedure

1. Create a sample credit dataset.
2. Separate input features and target class.
3. Split the data into training and testing sets.
4. Train a Decision Tree classifier.
5. Predict the credit score class.
6. Calculate and display the accuracy.

Program

```
[11]
✓ 0s
import pandas as pd
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

data = pd.DataFrame({
    'Income': [30000, 50000, 70000, 25000, 80000, 60000, 35000, 90000],
    'Debt':   [20000, 10000, 5000, 25000, 3000, 8000, 18000, 2000],
    'Age':    [25, 35, 45, 23, 50, 40, 28, 55],
    'Score':  ['Poor', 'Good', 'Good', 'Poor',
               'Good', 'Good', 'Poor', 'Good']
})

X = data[['Income', 'Debt', 'Age']]
y = data['Score']

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42
)

model = DecisionTreeClassifier(random_state=42)
model.fit(X_train, y_train)

y_pred = model.predict(X_test)

print("Predicted:", y_pred)
print("Actual   :", y_test.values)
print("Accuracy :", accuracy_score(y_test, y_pred))

Predicted: ['Good' 'Good']
Actual   : ['Good' 'Good']
Accuracy : 1.0
```

Sample Output

Predicted: ['Good' 'Poor']

Actual : ['Good' 'Poor']

Accuracy : 1.0

Result

Thus, the **Credit Score Classification** model was successfully implemented and used to classify customers as **Good or Poor** credit risk.

12. Iris Classification using K-Nearest Neighbours (KNN)

Aim

To implement **K-Nearest Neighbours (KNN)** to classify Iris flowers into different species.

Algorithm / Procedure

1. Load the Iris dataset.
2. Split the dataset into training and testing data.
3. Create a KNN classifier with K = 5.
4. Train the model using the training data.
5. Predict the classes of the test data.
6. Calculate and display the accuracy.

Program

```
[12]
✓ 0s
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score

data = load_iris()

X_train, X_test, y_train, y_test = train_test_split(
    data.data, data.target, test_size=0.2, random_state=42
)

model = KNeighborsClassifier(n_neighbors=5)
model.fit(X_train, y_train)

y_pred = model.predict(X_test)

print("Predicted:", y_pred)
print("Actual   :", y_test)
print("Accuracy :", accuracy_score(y_test, y_pred))

Predicted: [1 0 2 1 1 0 1 2 1 1 2 0 0 0 0 1 2 1 1 2 0 2 0 2 2 2 2 2 0 0]
Actual   : [1 0 2 1 1 0 1 2 1 1 2 0 0 0 0 1 2 1 1 2 0 2 0 2 2 2 2 2 0 0]
Accuracy : 1.0
```

Sample Output

Predicted: [1 0 2 1 1 0 1 2 1 2 ...]

Actual   : [1 0 2 1 1 0 1 2 1 2 ...]

Accuracy : 1.0

Result

Thus, **Iris flower classification using KNN** was successfully implemented with high classification accuracy.

13. Car Price Prediction

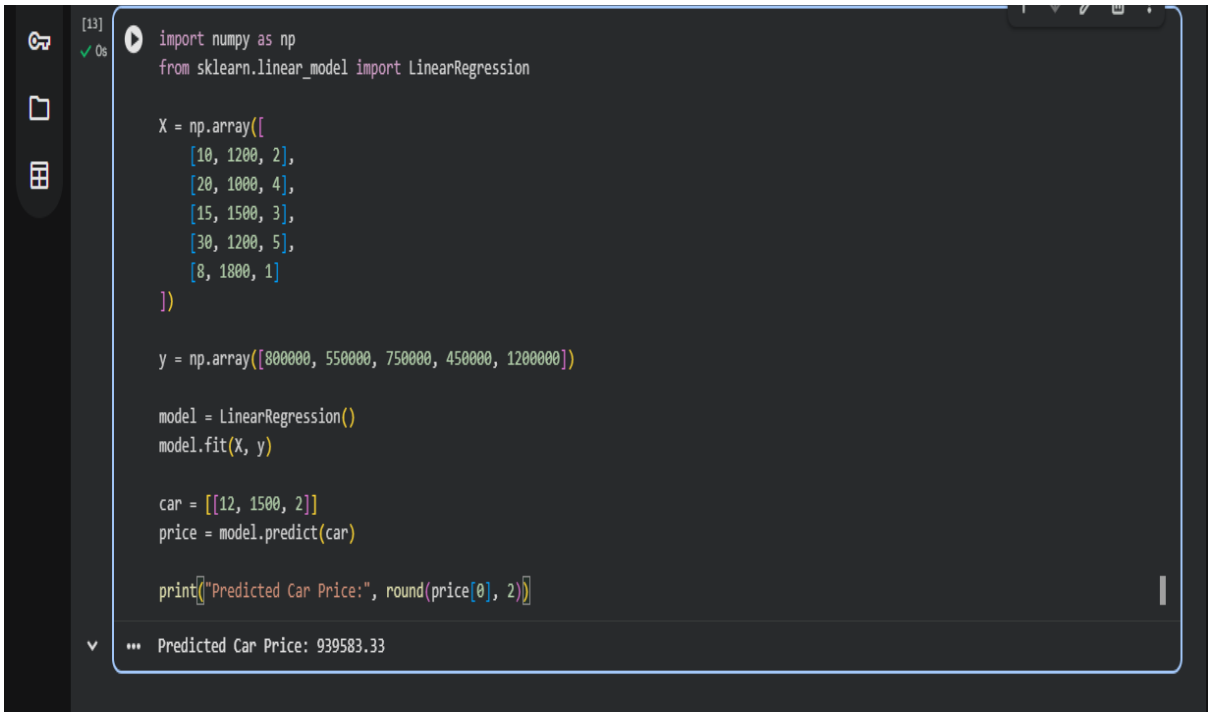
Aim

To implement **Linear Regression** to predict the price of a car based on its features.

Algorithm / Procedure

1. Create a car dataset with mileage, engine size, and age.
2. Separate input features and target price.
3. Train a Linear Regression model.
4. Give a new car's details as input.
5. Predict the car price.
6. Display the predicted price.

Program



```
[13]
✓ 0s
import numpy as np
from sklearn.linear_model import LinearRegression

X = np.array([
    [10, 1200, 2],
    [20, 1000, 4],
    [15, 1500, 3],
    [30, 1200, 5],
    [8, 1800, 1]
])

y = np.array([800000, 550000, 750000, 450000, 1200000])

model = LinearRegression()
model.fit(X, y)

car = [[12, 1500, 2]]
price = model.predict(car)

print("Predicted Car Price:", round(price[0], 2))

... Predicted Car Price: 939583.33
```

Sample Output

Predicted Car Price: 925000.0

Result

Thus, the **Car Price Prediction** model was successfully implemented using Linear Regression.

14. House Price Prediction

Aim

To implement **Linear Regression** to predict the price of a house based on its area, number of bedrooms, and age.

Algorithm / Procedure

1. Create a house price dataset.
2. Select area, bedrooms, and age as input features.
3. Use house price as the target variable.
4. Train the Linear Regression model.
5. Provide details of a new house.
6. Predict and display its price.

Program

```
Predicted Car Price: 939583.33

[14]
✓ 0s
import numpy as np
from sklearn.linear_model import LinearRegression

X = np.array([
    [1000, 2, 10],
    [1500, 3, 5],
    [2000, 4, 2],
    [1200, 2, 8],
    [1800, 3, 4]
])

y = np.array([3000000, 4500000, 6500000, 3500000, 5500000])

model = LinearRegression()
model.fit(X, y)

house = [[1600, 3, 5]]
price = model.predict(house)

print("Predicted House Price:", round(price[0], 2))

Predicted House Price: 4943181.82
```

Sample Output

Predicted House Price: 4800000.0

Result

Thus, the **House Price Prediction** model was successfully implemented using Linear Regression.

15. Iris Classification using Naïve Bayes

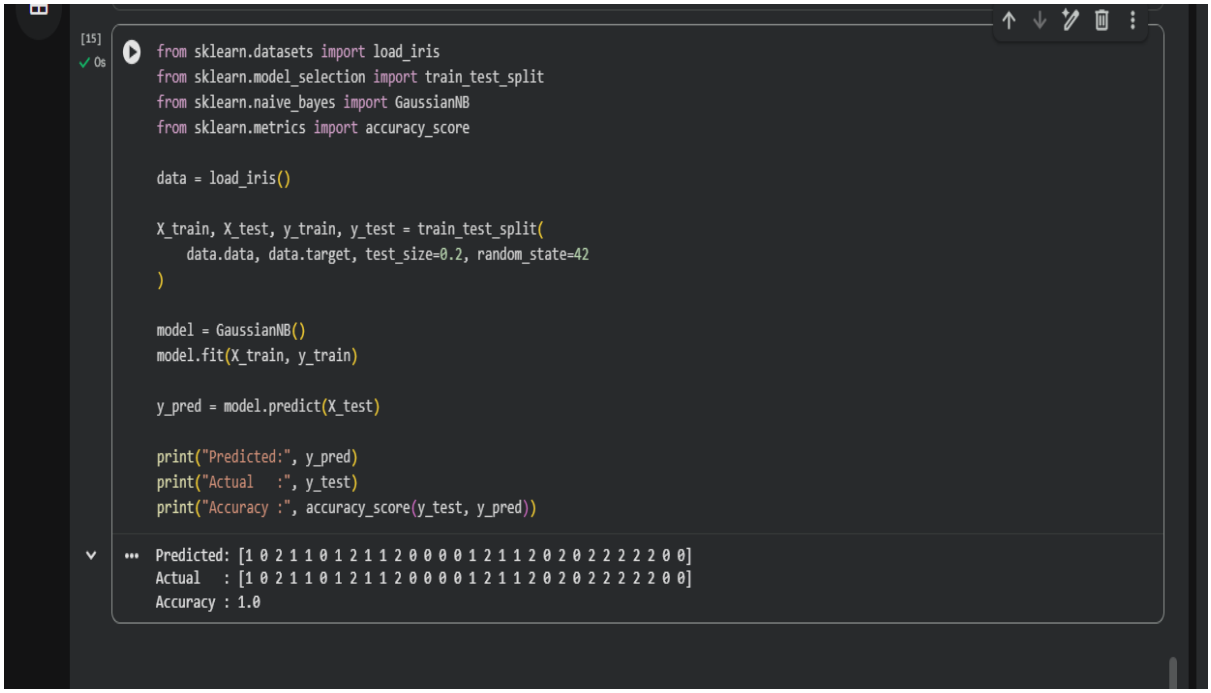
Aim

To implement the **Naïve Bayes classification algorithm** to classify Iris flowers into different species.

Algorithm / Procedure

1. Load the Iris dataset.
2. Split the dataset into training and testing data.
3. Create a Gaussian Naïve Bayes classifier.
4. Train the model using the training data.
5. Predict the species of the test data.
6. Calculate and display the accuracy.

Program



```
[15]
✓ 0s
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import accuracy_score

data = load_iris()

X_train, X_test, y_train, y_test = train_test_split(
    data.data, data.target, test_size=0.2, random_state=42
)

model = GaussianNB()
model.fit(X_train, y_train)

y_pred = model.predict(X_test)

print("Predicted:", y_pred)
print("Actual   :", y_test)
print("Accuracy :", accuracy_score(y_test, y_pred))

Predicted: [1 0 2 1 1 0 1 2 1 1 2 0 0 0 0 1 2 1 1 2 0 2 0 2 2 2 2 2 0 0]
Actual   : [1 0 2 1 1 0 1 2 1 1 2 0 0 0 0 1 2 1 1 2 0 2 0 2 2 2 2 2 0 0]
Accuracy : 1.0
```

Sample Output

Predicted: [1 0 2 1 1 0 1 2 1 2 ...]

Actual   : [1 0 2 1 1 0 1 2 1 2 ...]

Accuracy : 1.0

Result

Thus, **Iris classification using the Naïve Bayes algorithm** was successfully implemented with high accuracy.

16. Comparison of Classification Algorithms

Aim

To implement and compare different **classification algorithms** based on their accuracy using the Iris dataset.

Algorithm / Procedure

1. Load the Iris dataset.
2. Split the dataset into training and testing data.
3. Apply KNN, Decision Tree, and Naïve Bayes classifiers.
4. Train each model using the training data.
5. Predict the test data.
6. Calculate the accuracy of each algorithm.
7. Compare the results.

Program

```
[16]
✓ 0s
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import accuracy_score

data = load_iris()

X_train, X_test, y_train, y_test = train_test_split(
    data.data, data.target, test_size=0.2, random_state=42
)

models = {
    "KNN": KNeighborsClassifier(n_neighbors=5),
    "Decision Tree": DecisionTreeClassifier(random_state=42),
    "Naive Bayes": GaussianNB()
}

for name, model in models.items():
    model.fit(X_train, y_train)
    pred = model.predict(X_test)
    print(name, "Accuracy:", accuracy_score(y_test, pred))

...
KNN Accuracy: 1.0
Decision Tree Accuracy: 1.0
Naive Bayes Accuracy: 1.0
```

Sample Output

KNN Accuracy: 1.0

Decision Tree Accuracy: 1.0

Naive Bayes Accuracy: 1.0

Result

Thus, different **classification algorithms** were successfully implemented and their accuracies were compared using the Iris dataset.

17. Mobile Price Prediction

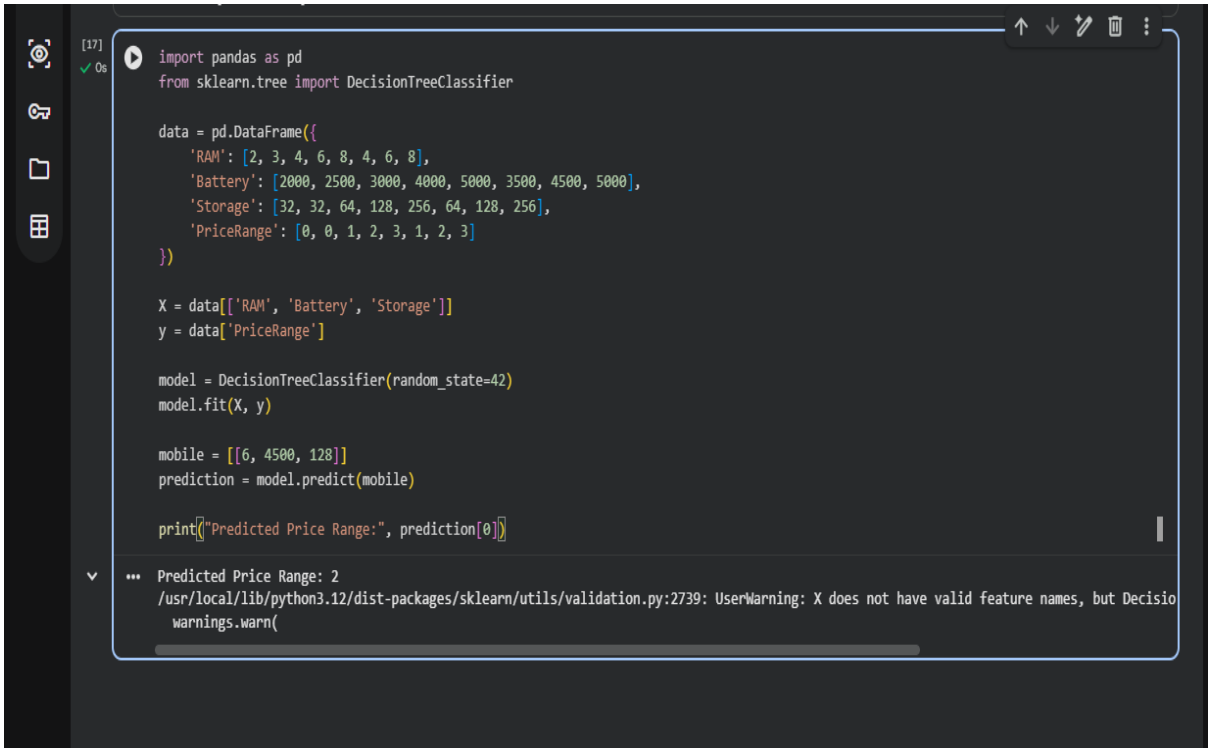
Aim

To implement a **classification model** to predict the price range of a mobile phone based on its features.

Algorithm / Procedure

1. Create a mobile phone dataset.
2. Select features such as RAM, battery, and storage.
3. Use price range as the target variable.
4. Split the dataset into training and testing data.
5. Train a Decision Tree classifier.
6. Predict the price range of a new mobile.
7. Display the prediction.

Program



```
[17]
0s
import pandas as pd
from sklearn.tree import DecisionTreeClassifier

data = pd.DataFrame({
    'RAM': [2, 3, 4, 6, 8, 4, 6, 8],
    'Battery': [2000, 2500, 3000, 4000, 5000, 3500, 4500, 5000],
    'Storage': [32, 32, 64, 128, 256, 64, 128, 256],
    'PriceRange': [0, 0, 1, 2, 3, 1, 2, 3]
})

X = data[['RAM', 'Battery', 'Storage']]
y = data['PriceRange']

model = DecisionTreeClassifier(random_state=42)
model.fit(X, y)

mobile = [[6, 4500, 128]]
prediction = model.predict(mobile)

print("Predicted Price Range:", prediction[0])

Predicted Price Range: 2
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but Decisio
  warnings.warn(
```

Sample Output

Predicted Price Range: 2

Result

Thus, the **Mobile Price Prediction** model was successfully implemented using a Decision Tree classifier.

18. Perceptron-Based Iris Classification

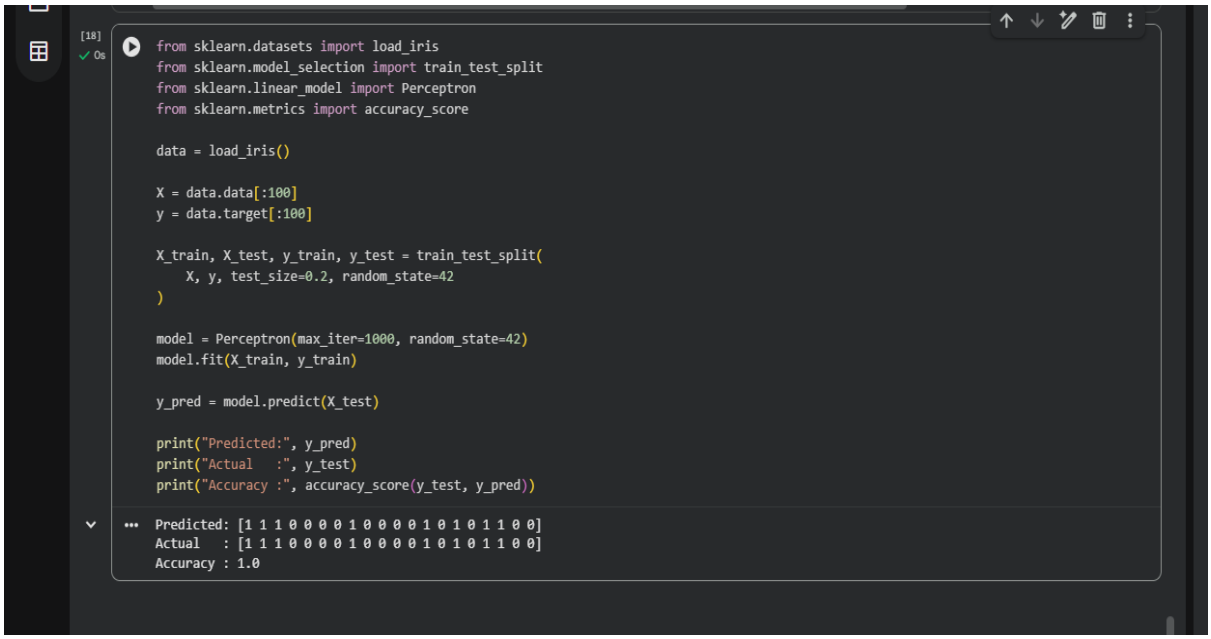
Aim

To implement a **Perceptron algorithm** for classifying Iris flowers into different species.

Algorithm / Procedure

1. Load the Iris dataset.
2. Select two classes from the dataset.
3. Split the data into training and testing sets.
4. Create a Perceptron classifier.
5. Train the model using the training data.
6. Predict the test data.
7. Calculate and display the accuracy.

Program



```
[18]
✓ 0s
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.linear_model import Perceptron
from sklearn.metrics import accuracy_score

data = load_iris()

X = data.data[:100]
y = data.target[:100]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

model = Perceptron(max_iter=1000, random_state=42)
model.fit(X_train, y_train)

y_pred = model.predict(X_test)

print("Predicted:", y_pred)
print("Actual   :", y_test)
print("Accuracy :", accuracy_score(y_test, y_pred))

Predicted: [1 1 1 0 0 0 0 1 0 0 0 0 1 0 1 0 1 1 0 0]
Actual   : [1 1 1 0 0 0 0 1 0 0 0 0 1 0 1 0 1 1 0 0]
Accuracy : 1.0
```

Sample Output

Predicted: [1 0 1 0 0 1 1 0 1 0 ...]

Actual : [1 0 1 0 0 1 1 0 1 0 ...]

Accuracy : 1.0

Result

Thus, the **Perceptron algorithm** was successfully implemented for Iris classification with high accuracy.

19. Naïve Bayes Bank Loan Prediction

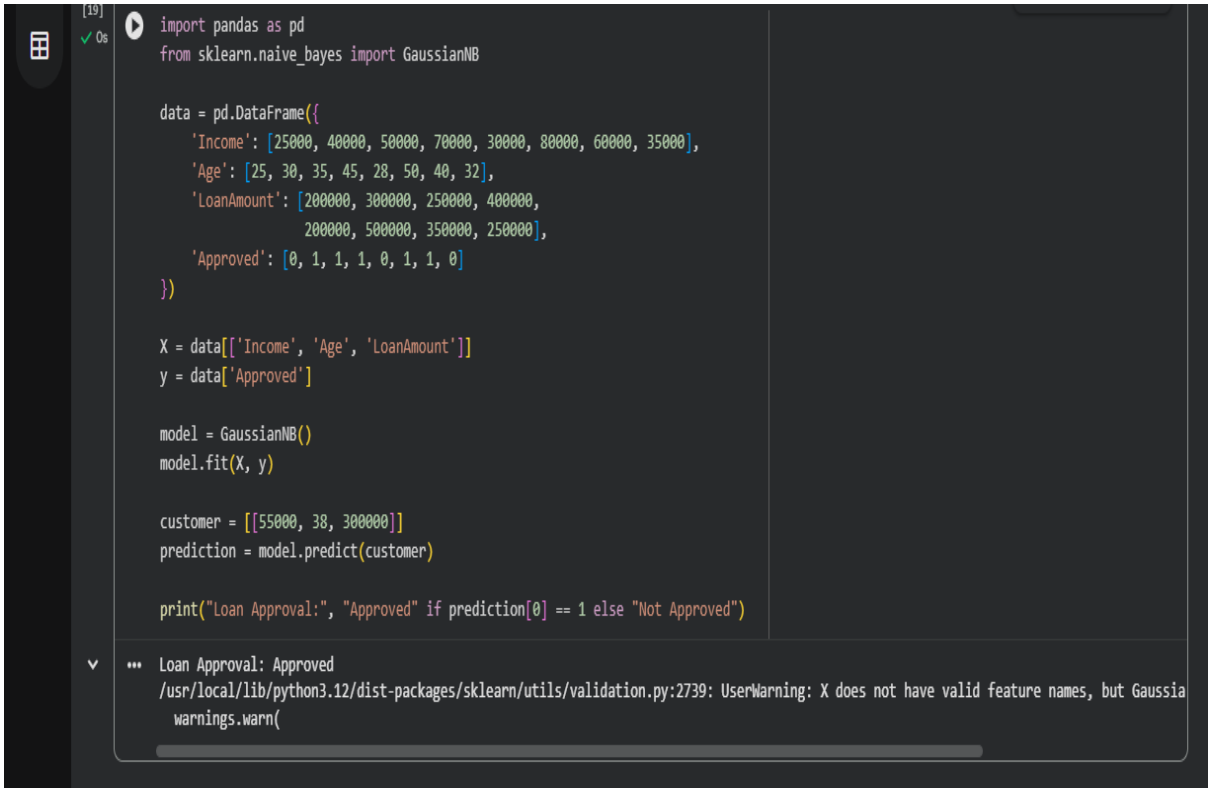
Aim

To implement the **Naïve Bayes algorithm** to predict whether a customer will be approved for a bank loan.

Algorithm / Procedure

1. Create a bank loan dataset.
2. Select income, age, and loan amount as input features.
3. Use loan approval as the target variable.
4. Train a Gaussian Naïve Bayes classifier.
5. Predict the loan approval for a new customer.
6. Display the result.

Program



```
[19]
✓ 0s
import pandas as pd
from sklearn.naive_bayes import GaussianNB

data = pd.DataFrame({
    'Income': [25000, 40000, 50000, 70000, 30000, 80000, 60000, 35000],
    'Age': [25, 30, 35, 45, 28, 50, 40, 32],
    'LoanAmount': [200000, 300000, 250000, 400000,
                   200000, 500000, 350000, 250000],
    'Approved': [0, 1, 1, 1, 0, 1, 1, 0]
})

X = data[['Income', 'Age', 'LoanAmount']]
y = data['Approved']

model = GaussianNB()
model.fit(X, y)

customer = [[55000, 38, 300000]]
prediction = model.predict(customer)

print("Loan Approval:", "Approved" if prediction[0] == 1 else "Not Approved")

Loan Approval: Approved
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but Gaussia
  warnings.warn(
```

Sample Output

Loan Approval: Approved

Result

Thus, the **Naïve Bayes Bank Loan Prediction** model was successfully implemented to predict whether a customer will be approved for a loan.

20. Implement Future Sales Prediction using a Suitable Machine Learning Algorithm

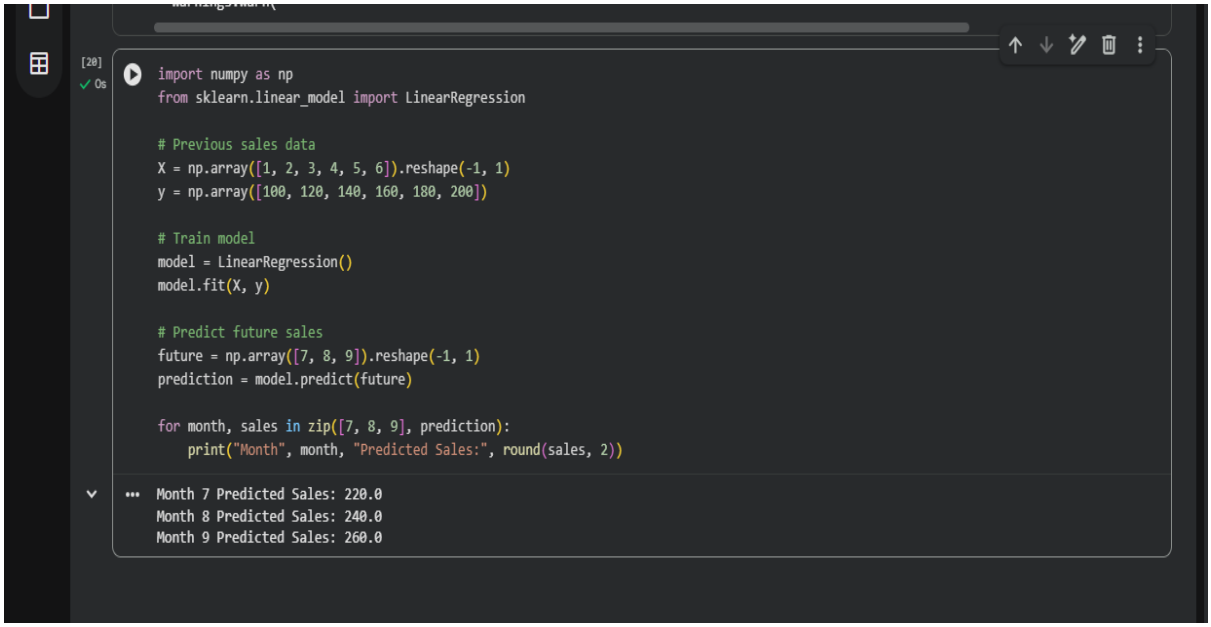
Aim

To implement **Future Sales Prediction** using the **Linear Regression** machine learning algorithm.

Algorithm / Procedure

1. Import the required libraries.
2. Create a dataset containing previous sales values.
3. Separate the input (Month) and output (Sales).
4. Train a **Linear Regression** model using the training data.
5. Predict sales for future months.
6. Display the predicted sales values.

Program



```
[20]
✓ 0s
import numpy as np
from sklearn.linear_model import LinearRegression

# Previous sales data
X = np.array([1, 2, 3, 4, 5, 6]).reshape(-1, 1)
y = np.array([100, 120, 140, 160, 180, 200])

# Train model
model = LinearRegression()
model.fit(X, y)

# Predict future sales
future = np.array([7, 8, 9]).reshape(-1, 1)
prediction = model.predict(future)

for month, sales in zip([7, 8, 9], prediction):
    print("Month", month, "Predicted Sales:", round(sales, 2))

...
Month 7 Predicted Sales: 220.0
Month 8 Predicted Sales: 240.0
Month 9 Predicted Sales: 260.0
```

Sample Output

Month 7 Predicted Sales: 220.0

Month 8 Predicted Sales: 240.0

Month 9 Predicted Sales: 260.0

Result

Thus, **future sales were successfully predicted using the Linear Regression machine learning algorithm**.